

Exploratory Data Analysis on Spotify Tracks Dataset Utilizing Vectorized NumPy Operations

By: Nayla Putri Ghaisani (AI18)

Submission:

- Notebook:

<https://colab.research.google.com/drive/1G1Yz3TAUcee91Wzh2XFSvOGk5kERC19?usp=sharing>

- Github upload:

<https://github.com/nylghaisani/GDGoC/tree/1be3c4f995c3dc069a12ce8e12b3ea2a9cb8d879/Artificial%20Intelligence>

Part 1: Overview

The year is 2020, and the digital music streaming revolution has completely transformed how humanity consumes audio. To organize millions of tracks and recommend the perfect song, Spotify the world's leading audio platform relies on intricate algorithms and hidden acoustic features.

But amidst this massive sea of data, a challenge arises. With tens of thousands of songs competing for attention across various playlists, uncovering what truly makes a track explode in popularity or fade into obscurity remains a complex puzzle. The fate of a song's success hangs in the balance of its underlying metrics.

This analysis challenges us to use audio records including danceability, energy, acousticness, and genre metadata to dissect and understand the DNA of a hit song. By exploring patterns in the data using pure vectorized mathematics, we aim to uncover the hidden factors that drive listener engagement.

- Understanding the role of audio characteristics (energy, valence, tempo) in overall track popularity.
- Exploring how different genres experience varying levels of success and volatility in the streaming market.
- Identifying the audio fingerprints of top artists and their correlation with consistent chart-topping performance.

Beyond just crunching numbers, this project immerses us in the rhythm of data science where analytical coding becomes the key to unraveling the secrets behind the music we listen to every day on Spotify.

Part 2: Methodology & Exploration Journey

Throughout this project, I adopted the Vibe Coding approach described in the module. However, instead of treating the AI as an autopilot that writes entire scripts from scratch, I utilized it strictly as a technical co-pilot. I acted as the architect of the analysis structuring the logic and pipeline myself and consulted the AI to bridge syntax gaps, explain mathematical implementations, and optimize specific Pandas/NumPy operations. I rigorously evaluated, modified, and assembled the AI's snippet suggestions into the final working codebase. Below is a breakdown of this collaborative workflow from Stage 1 to Stage 3.

Stage 1: Setup and Initial Inspection

Objective: Load the Spotify dataset and ensure data integrity by removing duplicate tracks dynamically.

AI Prompt Engineered: "I am cleaning a Spotify dataset using Pandas. What is the most efficient method to drop duplicate rows based strictly on a specific column (like 'track_id') while ensuring the first occurrence is kept in the DataFrame?"

Implementation & Reflection: Instead of asking for a full script, I asked for the specific method. The AI introduced me to the `subset` and `keep='first'` parameters of `drop_duplicates()`. I then independently wrote the data loading sequence, implemented the cleaning logic, and added the `.shape[0]` comparisons to mathematically verify the row reduction.

Stage 2: Genre and Popularity Analysis

Objective: Group the data by genre, calculate the variance in popularity, and programmatically extract the top genre without using inefficient Python loops.

AI Prompt Engineered: "I have a Pandas Series containing the variance of track popularity grouped by genre. How can I programmatically extract both the index name (the genre) and the actual value of the highest variance without writing a for-loop?"

Implementation & Reflection: I had already written the `.groupby()['track_popularity'].var()` logic myself. The challenge was extracting the peak value efficiently. The AI suggested using `.idxmax()` to get the genre string and `.max()` to get the float value. I reviewed this approach and integrated it into my own dictionary formatting, ensuring the solution was both elegant and Pythonic.

Stage 3: NumPy Analysis (Strictly Vectorized)

Objective: Perform a Min-Max scaling normalization and isolate the two most positively correlated audio features using pure NumPy, avoiding `scipy` or `pandas.corr()`.

AI Prompt Engineered: "Explain the mathematical formula for Min-Max scaling and how to implement it across a 2D NumPy array using broadcasting. Also, after computing a correlation matrix with `np.corrcoef`, how do I extract the row and column indices of the highest value while ignoring the diagonal of 1s?"

Implementation & Reflection: Because this stage strictly forbade high-level statistical libraries, I needed to ensure the mathematical foundation was flawless. The AI explained the broadcasting formula:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$
 I wrote the array conversions and implemented this vectorized formula manually.

For the correlation matrix, the AI provided a brilliant technique using `np.triu_indices_from(corr_matrix, k=1)` to isolate the upper triangle of the matrix (ignoring the diagonal). I used the Code Explanation pattern to fully understand this function before implementing it. I then wrote the final logic to map those numeric indices back to my `audio_cols` list.

Part 3: AI Integration & OOP Implementation

3.1 AI-Assisted Development & Docstring Generation

In alignment with modern software engineering practices, specifically the concept of Vibe Coding, an AI assistant (Gemini) was utilized to streamline the documentation process. Writing comprehensive documentation from scratch can be time-consuming. However, by acting as the prompt architect, I

directed the AI to generate standardized Google-style docstrings for the entire pipeline, ensuring proper type hinting throughout the code.

To ensure the AI acted as an efficient co-pilot rather than an autopilot, a highly specific and context-driven prompt strategy was engineered. Instead of requesting broad, generic code blocks, the prompt was structured with strict constraints to generate exceptionally brief, single-line summaries for each method. The primary goal was to maximize code readability while enforcing explicit Python Type Hints (such as `Dict[str, Any]` and `List[str]`).

I manually reviewed the type hints to guarantee compatibility with the standard typing library environment inside Google Colab. Furthermore, I aggressively trimmed the descriptions to ensure they were strictly to the point, eliminating generic AI filler text and finalizing a production-grade class structure.

3.2 Object-Oriented Refactoring (SpotifyAnalyzer)

To transform a series of disconnected Exploratory Data Analysis (EDA) scripts into a robust, reusable production pipeline, the entire codebase was refactored into a single Python class named `SpotifyAnalyzer`. This class encapsulates every analytical phase completed in the previous stages into dedicated, distinct methods:

- Data Cleaning: Reusable duplicate removal using `clean_data()`.
- Statistical Logic: Dynamic variance computation via `analyze_genre_variance()`.
- Vectorized Math: Independent Min-Max scaling and advanced correlation mapping via `analyze_correlations()`.
- Boolean Masking: Evaluating high-energy outlier tracks via `evaluate_high_energy()`.

By wrapping the logic into a class architecture, the entire execution flow is centralized. The `generate_report()` method orchestrates the pipeline sequentially, creating an integrated data flow where each method passes validated states to aggregate the final results into a single, nested Python dictionary.

3.3 Production Pipeline Verification & Report Output

To satisfy the module's strict requirement for zero external setup, the class includes an autonomous execution block (`if __name__ == "__main__":`). When executed, it pulls the raw streaming data directly from the TidyTuesday repository, instantly handles cleaning, runs all mathematical evaluations, and outputs a single comprehensive report payload serialized as a clean, indented JSON layout.

Part 4: Reference

[1] T. Mock, "TidyTuesday: Spotify Songs," *GitHub - rfordatascience*, Jan. 2020. [Online]. Available: <https://github.com/rfordatascience/tidytuesday/blob/master/data/2020/2020-01-21/readme.md>.

Accessed on: Jun. 21, 2026.

[2] Pandas Developers, "pandas.DataFrame.dropna," *pandas documentation*. [Online]. Available: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>. Accessed on: Jun. 21, 2026.

[3] GeeksforGeeks, "Drop empty columns in Pandas," *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/python/drop-empty-columns-in-pandas/>. Accessed on: Jun. 24, 2026.

[4] NumPy Developers, "NumPy learning resources," *NumPy*. [Online]. Available: <https://numpy.org/learn/>. Accessed on: Jun. 28, 2026.

[5] Stanford University CS231n, "Python Numpy Tutorial," *CS231n: Convolutional Neural Networks for Visual Recognition*. [Online]. Available: <https://cs231n.github.io/python-numpy-tutorial/>. Accessed on: Jun. 29, 2026.

[6] GitHub, "GitHub Copilot documentation," *GitHub Docs*. [Online]. Available: <https://docs.github.com/en/copilot>. Accessed on: Jul. 1, 2026.